



data-admin - Manage Access Control

Accountable	Antoine Chapelet (J) Jitesh Pillai
Consulted	Antoine Chapelet
Domain	DE - Data Engineering
Responsible	Antoine Chapelet (J) Jitesh Pillai
Role	data-admin.Owner
Service	Snowflake
Status	Done

Constraints

Qima needs

1. Follow **RBAC** principles: Organize roles with layers (User → Functional role → Access role → Data)
2. Follow **medaillon architecture** (Bronze, Silver, Gold)
3. Support **domains and dubdomains**
4. Create access with **table or column precision**
5. **Unified model**: All databases, schema, objects follow the same model
 - Easier to understand, manage, control
2. **Extensible model**: While unified, the model should be easily adaptable to different and future use cases
3. Managable transition from existing model
4. Simple, transparent, auditable

Snowflake constraints

- **Database owner** role (here F_DOMAIN_DBA)
 - CREATE, MANAGE, OWN all database roles.
 - Grant privileges to database roles.
 - Only on objects within the database
 - Can be any privilege, including OWNERSHIP
 - granting a database role to an account role implicitly grants the USAGE privilege on the database that contains the database role to that account role
- **Database roles**
 - Cannot be used directly by a user, only via a functional role
 - Docs
 - <https://docs.snowflake.com/en/user-guide/security-access-control-considerations#managing-database-object-access-using-database-roles> - **Managing database object access using database roles**
- Grant custom roles to SYSADMIN
 - <https://docs.snowflake.com/en/user-guide/security-access-control-considerations#example>
- All shemas with **managed access**

- Docs
 - <https://docs.snowflake.com/en/user-guide/security-access-control-considerations#centralizing-grant-management-using-managed-access-schemas> - **Centralizing grant management using managed access schemas¶**
 - <https://docs.snowflake.com/en/user-guide/security-access-control-configure#creating-managed-access-schemas> - **Creating managed access schemas¶**
- Use **future grants** for all object grants
 - Docs:
 - <https://docs.snowflake.com/en/user-guide/security-access-control-considerations#simplifying-grant-management-using-future-grants> - **Simplifying grant management using future grants¶**
 - <https://docs.snowflake.com/en/user-guide/security-access-control-configure#label-granting-future-privs-on-schema-objects> - **Assigning future grants on objects¶**

Factorizations

1. Equivalence between “SQL Sublanguage” and “Privilege level” (Access Roles)

SQL Sublanguage	Privilege level
DCL (Data Control Language)	A (Admin)
DDL (Data Definition Language)	O (Own)
DML (Data Manipulation Language)	W (Write)
DQL (Data Query Language)	R (Read)

- Commonly used in software development

2. Equivalence between “RBAC role type” and “Snowflake role scope” (All Roles)

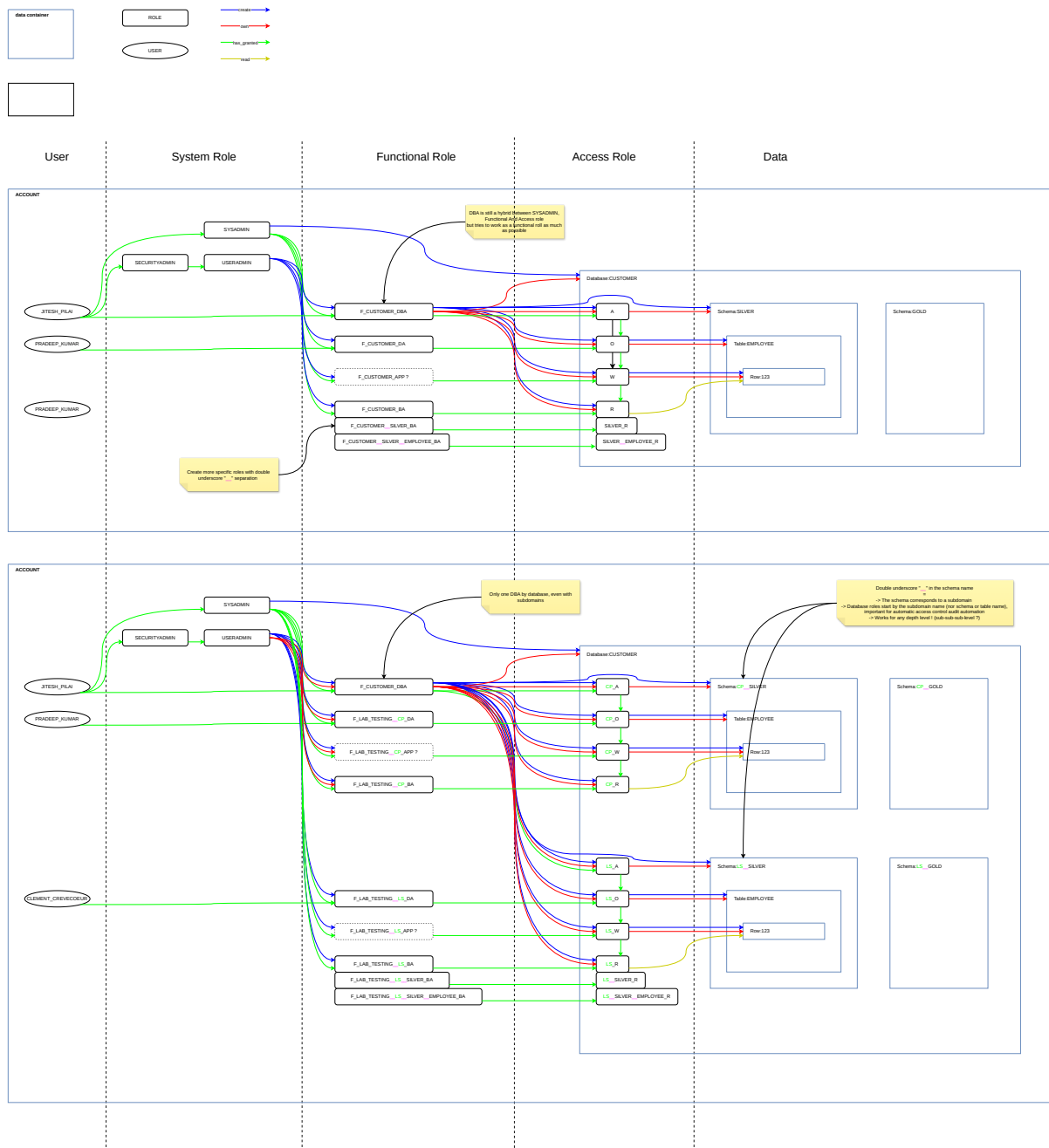
RBAC role type	Snowflake role scope
Functional role	Account role
Access Role	Database role

- Limits the blast radius of access roles
- Maps well with Snowflake architecture
- Recommended by the community ?
 - <https://medium.com/snowflake/role-based-access-control-rbac-with-snowflake-database-roles-425f560c1873>

3. Access roles follow a hierarchy to inherit privileges: **DBA** (DB Admin) ← **A** (Admin) ← **O** (Own) ← **W** (Write) ← **R** (Read)

- **Why:**
 - Simplify access and functional role creation
 - It can be disabled easily
- **Why not:**
 - to have a finer access control (eg: migrate a table schema without getting access to its data)
 - Disabled in regulated environments

Architecture Schema



Todo

- ☐ Remove direct use of system roles (ACCOUNTADMIN, SYSADMIN, SECURITYADMIN, USERADMIN) → Still available but the platform should work without needing to use it
- ☐ 99% of operations should work only with functional roles (F_{DOMAIN}_DBA, F_DE, F_DA, etc ...)
- ☐ Create a specific user for each F_{DOMAIN}_DBA @Jitesh Pillai
- ☐ 1% of operations can use only dedicated users to use system roles (SYSADMIN_{USERNAME})
- ☐ at least 2 dedicated users by system role
- ☐ Create sql queries to monitor the access control structure conformity
- ☐ View follows always the same structure to be stacked with the other views

- ☐ Add BA roles
- ☐ Add ABAC roles

Matrix

Status	Name	Detail	By
?	To Confirm	Needs debate or ??? before creation	Anyone
○	To Create	Ready to create	Access control manager
✗	To Delete	Existing role needs to be deleted. Remove the text	Anyone
✓	Done	The role exists in snowflake with the corresponding relations	Access control manager
☑	Done - Immutable		-

Level	A (Admin)	O (Own)
	Manage who can do what	Define the data container/structure
SQL Sublanguage WikipediaSQL	DCL (Data Control Language) • https://en.wikipedia.org/wiki/Data_control_language	DDL (Data Definition Language) • https://en.wikipedia.org/wiki/Data_definition_language
Commands	- GRANT - give permissions - REVOKE - remove permissions	- CREATE - create a new object - ALTER - modify an existing object's property - DROP - remove an object - UNDROP - recover a dropped object (via Time Travel) - TRUNCATE - remove all rows (= DROP + CREATE) - RENAME - rename an object - COMMENT - add or update a comment on an object
Typical user example	Security team (manage user access, LDAP, Active directory)	- Database admin (designing schemas, creating tables, managing indexes, optimizing storage, handling backups, tuning performance) - Migration scripts CI/CD pipelines to apply d (eg: from python alembic)
Access Roles - Standard model: - Bronze layer database: RAW, RAW_DEV - Domain database without subdomains: FINANCE, PRODUCT, SANDBOX, ADMIN		
owned by: <code>SHOW GRANTS OF ROLE IDENTIFIER(\$role_cd); ?</code>	SYSADMIN	SYSADMIN
code: <code>IDENTIFIER(\$role_cd); ?</code>	• {DOMAIN}.A	• {DOMAIN}.O
owner of: <code>SHOW GRANTS TO ROLE IDENTIFIER(\$role_cd); ?</code>	- A: CREATE, OWN, MANAGE ACCESS on all schemas in the domain database: - SANDBOX.{USER} - RAW.{SOURCE} - RAW_DEV.{SOURCE} {DOMAIN}.SILVER {DOMAIN}.GOLD <code>CREATE SCHEMA WITH MANAGED ACCESS</code>	- O: CREATE, OWN all objects in standard schema - RAW.{SOURCE}.{OBJECT} - RAW_DEV.{SOURCE}.{OBJECT} {DOMAIN}.SILVER.{OBJECT} {DOMAIN}.GOLD.{OBJECT} <code>GRANT USAGE ON SCHEMA {SOURCE} TO DATABASE ROLE {DOMAIN}.O</code> For each object type: <code>GRANT OWNERSHIP ON ALL {OBJECT} IN SCHEMA {SOURCE} TO DATABASE ROLE {DOMAIN}.O COPY CURRENT GRANTS</code> <code>GRANT OWNERSHIP ON FUTURE {OBJECT} IN SCHEMA {SOURCE} TO DATABASE ROLE {DOMAIN}.O;</code>

Level	A (Admin)	O (Own)
Access Roles - Subdomain model (extension): - Domain database with subdomains: AUDIT, LAB_TESTING - Can be easily merged into standard model after normalization		
Role	• {DOMAIN}_O (same as base model)	• {DOMAIN}.{SUBDOMAIN}_O
Grants	OWN all subdomain schemas in the domain database: {DOMAIN}.{SUBDOMAIN}_SILVER {DOMAIN}.{SUBDOMAIN}_GOLD	OWN all objects in subdomain schema: {DOMAIN}.{SUBDOMAIN}_SILVER.{OBJECT} {DOMAIN}.{SUBDOMAIN}_GOLD.{OBJECT}
Access Roles - By Domain		
SANDBOX ?	• SANDBOX.A ?	• SANDBOX.O ?
ADMIN ○	• ADMIN.A ○	• ADMIN.O ○
RAW ✓	• RAW.A ✓	• RAW.O ✓
RAW_DEV ✓	• RAW_DEV.A ✓	• RAW_DEV.O ✓
AUDIT ✓	- AUDIT.CP_A ✓ - AUDIT.FOOD_A ✓	- AUDIT.CP_O ✓ - AUDIT.FOOD_O ✓
PRODUCT ✓	• PRODUCT.A ✓	• PRODUCT.O ✓
FINANCE ✓	• FINANCE.A ✓	• FINANCE.O ✓
LAB_TESTING ✓	- LAB_TESTING.CP_A ✓ - LAB_TESTING.LS_A ✓	- LAB_TESTING.CP_O ✓ - LAB_TESTING.LS_O ✓
Access Roles - By Warehouse		
Functional Roles - System Roles		
ACCOUNTADMIN ✓		
SYSADMIN ✓ ✗? ✗? - S_SYSADMIN_JITESH_PILLAI ○? (no more direct use) - S_SYSADMIN_VICTOR_GRAPPE ○?	• ALL ✓	• ALL ✓
USERADMIN ✓ ✗? ✗? - S_USERADMIN_JITESH_PILLAI		

Level	A (Admin)	O (Own)
○? (no more direct use) S_USERADMIN_VICTOR_GRAPPE ○?		
Functional Roles - ETL		
F_RAW_DBA ✓ ✓ ✓	RAW.A ✓	
F_RAW_DEV_DBA ✓ ✓ ✓	RAW_DEV.A ✓	
F_AUDIT_DBA ✓ ✓ ✓	AUDIT.A ✓	
F_PRODUCT_DBA ✓ ✓ ✓	PRODUCT.A ✓	
F_FINANCE_DBA ✓ ✓ ✓	FINANCE.A ✓	
F_LAB_TESTING_DBA ✓ ✓ ✓	LAB_TESTING.A ✓	
Functional Roles - DE (Data Engineer)		
F_DE ✓ ✓ ✓ ✓ ✓		RAW.O ✓
F_DE_DEV ✓ ✓ ✓ ✓ ✓		RAW_DEV.O ✓
Functional Roles - ETL		
F_ETL ✓ (data-import-prod) ✓ ✓		RAW.O ✓
F_ETL_DEV ✓ (data-import-dev) ✓		RAW_DEV.O ✓
F_ETL_USER ? (modify the ETL configuration) → add to DA ?		
F_ETL_USER_DEV ? (modify the ETL configuration)		
Functional Roles - DA (Data Analyst)		
F_DA ✓ ✓ ✓		- AUDIT.CP_O ✓ - AUDIT.FOOD_O ✓ - FINANCE.O ✓

Level	A (Admin)	O (Own)
		- LAB_TESTING.CP_O ✓ - LAB_TESTING.LS_O ✓ - PRODUCT.O ✓
F_AUDIT_CP_DA ✓ ✓		AUDIT.CP_O ✓
F_AUDIT_FOOD_DA ✓ ✓		AUDIT.CP_O - AUDIT.FOOD_O ✓
F_FINANCE_DA ✓ ✓ ✓		FINANCE.O ✓
F_LAB_TESTING_CP_DA ✓ ✓		LAB_TESTING.CP_O ✓
F_LAB_TESTING_LS_DA ✓ ✓		LAB_TESTING.LS_O ✓

Level	A (Admin)	O (Own)
F_PRODUCT_DA  • 		• PRODUCT.O 
• SO_DOMAIN		
Functional Roles - BA (Business Analyst)		
F_BA		